

Pensamento Lógico Computacional com Python

Aula 04

Prof.ª Larissa

Sumário

- Funções
 - Argumento
 - Retorno
 - Parâmetro
 - Valor/referência
- Estruturas de dados
 - Dicionários
 - for percorrendo dicionários
 - Dicionários como argumentos

Funções: introdução

Ao longo de nossas aulas, já usamos diversas funções em Python. Por exemplo, a função `max()` pode receber dois **argumentos** numéricos como entrada e **retorna** o maior deles:

```
>>> max(4, 7)
```

```
7
```

Já a função `sum()` pode receber uma lista de números como **argumento** e **retorna** o somatório desses valores:

```
>>> sum([4, 5, 6, 7])
```

```
22
```

Algumas funções podem, ainda, ser chamadas sem argumentos:

```
>>> entrada = input()
```

Funções: introdução

Em geral, uma função recebe 0 ou mais argumentos de entrada e retorna um resultado. Quando chamada, uma função executa um bloco de código que, muitas vezes, é desconhecido pelo próprio programador. Isso simplifica o desenvolvimento de programas e permite o reuso do código.

Qual é a saída do *script* a seguir?

```
→ 1. def quadrado(num):  
→ 2.     return num ** 2  
→ 3.  
→ 4. a = 3  
→ 5. a_quadr = quadrado(a)  
→ 6. print(a_quadr)
```

Linha 4: variável a recebe valor 3.

Linha 5: função quadrado() é chamada com o argumento a (que vale 3).

Linha 1: parâmetro num recebe o argumento (3).

Linha 2: num é elevado ao quadrado (9) e seu valor é retornado para “quem chamou a função”.

Linha 5: variável a_quadr recebe o retorno da função (9).

Linha 6: finalmente, o valor de a_quadr é impresso.

Funções: def, parâmetros e retorno

Uma função é definida por meio da palavra-chave **def**, seguida pelo nome da função, **parâmetros** opcionais e um corpo que contém as instruções a serem executadas. Ao encapsular uma lógica específica dentro de uma função, é possível chamar esse bloco repetidamente em diferentes partes de um programa, evitando a duplicação de código e facilitando a manutenção.

def: palavra-chave de definição de função

Parâmetro(s): variáveis locais que “aguardam a chegada” de argumentos. Separados entre si por vírgula. Parâmetros não são obrigatórios.

```
def quadrado(num):  
    return num ** 2
```

Retorno: “resposta” que a função dá a quem a chamou. O retorno não é obrigatório.

Funções: introdução

Quando passamos valores numéricos ou *strings* como argumentos, criamos uma “cópia” de seus valores nas variáveis que atuam como parâmetros (passagem por valor). Com isso, mudanças que são realizadas dentro das funções não se refletem na variáveis fora dela.

```
1. def troca(a, b):  
2.     3 temp = a  
3.     7 a = b  
4.     3 b = temp  
5.  
6. a = 3  
7. b = 7  
8. troca(a, b)  
9. print(f"a = {a}, b = {b}")
```

Esse código “não funciona” para trocar valores porque Python trata tipos imutáveis (como numéricos e *strings*) como passados por valor, e as alterações ficam restritas ao escopo da função `troca()`.

```
a = 3, b = 7
```

Funções: introdução

Podemos fazer um “retorno múltiplo”, retornando uma tupla (uma estrutura de valores separados por vírgula). Observe.

```
1.  def troca(a, b):  
2.      3 temp = a  
3.      7 a = b  
4.      3 b = temp  
5.      return a, b  
6.  
7.  a = 3  
8.  b = 7  
9.  a, b = troca(a, b)  
10. print(f"a = {a}, b = {b}")
```

Esse código “funciona” para trocar valores porque os valores trocados foram retornados para fora do escopo da função `troca()`. Com isso, as variáveis globais `a` e `b` receberam os valores retornados.

```
a = 7, b = 3
```

Funções: lista como argumento

Quando passamos uma **lista** como argumento, passamos seu endereço de memória para a variável que atua como parâmetro (passagem por referência). Com isso, mudanças que são realizadas dentro das funções se refletem na lista que há fora de seu escopo.

```
1. def adiciona_um(lista):  
2.     for i in range(len(lista)):  
3.         lista[i] += 1  
4.  
5. numeros = [2, 3, 4, 5]  
6. adiciona_um(numeros)  
7. print(numeros)
```

	[2, 3, 4, 5]
Índice:	0 1 2 3
	↓
	[3, 4, 5, 6]

[3, 4, 5, 6]

Estruturas de dados: Dicionários - introdução

Em Python, um **dicionário** é uma coleção de objetos cujo identificador pode ser definido por nós, ao invés de termos um indicador numérico fixo, como acontece com os índices das listas. Um dicionário contém **itens** que são pares de **chave: valor**. O formato geral da expressão avaliada como um objeto dicionário é:

`{<chave 1>:<valor 1>, <chave 2>:<valor 2>, ..., <chave i>:<valor i>}`

As chaves podem ser quaisquer valores imutáveis (numéricos ou *strings*), e os valores podem ser de qualquer tipo. Por exemplo, podemos criar o dicionário `alunos`, contendo dois pares chave:valor, com o comando a seguir.

```
>>> alunos = {"35TA2F": "Ana Santos", "0F83RG": "João Torres"}
```

Nesse caso, os valores de RA são as chaves que identificam cada aluno. Os itens de um dicionário não têm ordem específica entre si.

Estruturas de dados: Dicionários - alguns operadores

Vamos testar alguns operadores e funções usados para manipular dicionários. Alguns operadores de listas também opera sobre dicionários.

```
>>> dias = {'Seg': 'Segunda', 'Ter': 'Terça', 'Qua': 'Quarta'}
```

```
>>> type(dias)
```

```
<class 'dict'>
```

```
>>> dias['Qua']
```

```
'Quarta'
```

```
>>> dias['Qui'] = 'Quinta'
```

```
>>> dias
```

```
{'Seg': 'Segunda', 'Ter': 'Terça', 'Qua': 'Quarta', 'Qui': 'Quinta'}
```

```
>>> len(dias) #Retorna nº de pares chave:valor do dicionário
```

```
4
```

Estruturas de dados: Dicionários - alguns operadores

Continuando...

```
>>> 'Qua' in dias #Verifica se o objeto é uma chave no dict.
```

```
True
```

```
>>> 'Quarta' not in dias
```

```
True
```

```
>>> del dias['Qui'] #Deleta o item cuja chave foi enviada.
```

```
>>> dias
```

```
{'Seg': 'Segunda', 'Ter': 'Terça', 'Qua': 'Quarta'}
```

```
>>> max(dias) #Maior chave em ordem numérica ou alfabética
```

```
'Ter'
```

Estruturas de dados: Dicionários - alguns métodos

Os **métodos** de dicionários são funções chamadas pelo operador de ponto para operar sobre dicionários específicos. Veremos alguns, a seguir.

```
>>> alunos = {'RA01': 'Maria Clara', 'RA02': 'Marco Antônio'}
>>> alunos.pop('RA01') #Remove item cuja chave foi enviada e retorna
'Maria Clara'

>>> alunos
{'RA02': 'Marco Antônio'}
>>> alunos2 = {'RA03': 'Sara Flores'}
>>> alunos.update(alunos2) #Acrescenta a alunos os itens de alunos2

>>> alunos
{'RA02': 'Marco Antônio', 'RA03': 'Sara Flores'}
```

Estruturas de dados: Dicionários - alguns métodos

Continuando...

```
>>> alunos.keys() #Retorna as chaves do dict alunos
```

```
dict_keys(['RA02', 'RA03'])
```

```
>>> alunos.values() #Retorna os valores do dict alunos
```

```
dict_values(['Marco Antônio', 'Sara Flores'])
```

```
>>> alunos.items() #Retorna os itens do dict alunos
```

```
dict_items([('RA02', 'Marco Antônio'), ('RA03', 'Sara Flores')])
```

```
>>> alunos.get('RA03') #Retorna o valor associado à chave recebida
```

```
'Sara Flores'
```


Estruturas de dados: for percorrendo dicionários

Podemos utilizar o laço for para percorrer dicionários. Nesse caso, é comum o uso dos métodos `keys()`, `values()` e `items()`. Observe os exemplos.

```
1.  frutas = {'maçã':3.50, 'uva':6.80, 'romã':9.30}
2.  for chave in frutas:      #frutas equivale a frutas.keys()
3.      print(f"kg da {chave} custa R${frutas[chave]:0.2f}")
```

```
kg da maçã custa R$3.50
kg da uva custa R$6.80
kg da romã custa R$9.30
```

Estruturas de dados: for percorrendo dicionários

Podemos utilizar o laço for para percorrer dicionários. Nesse caso, é comum o uso dos métodos `keys()`, `values()` e `items()`. Observe os exemplos.

```
1. frutas = {'maçã':3.50, 'uva':6.80, 'romã':9.30}
2. for chave, valor in frutas.items():
3.     print(f"kg da {chave} custa R${valor:0.2f}")
4. total = sum(frutas.values())
5. print(f"Total: R${total:0.2f}")
```

```
kg da maçã custa R$3.50
kg da uva custa R$6.80
kg da romã custa R$9.30
Total: R$19.60
```

Funções: dicionário como argumento

Quando passamos um **dicionário** como argumento, passamos seu endereço de memória para a variável que atua como parâmetro (passagem por referência). Com isso, mudanças que são realizadas dentro das funções se refletem no dicionário que há fora de seu escopo.

```
1. def aumentar_precos(produtos, aumento):  
2.     for item in produtos:  
3.         produtos[item] = produtos[item] + aumento  
4.  
5. produtos = {"pão": 5.9, "leite": 4.5, "queijo": 8.0}  
6.  
7. aumentar_precos(produtos, 1)  
8. print(f"Após aumento: {produtos}")
```

```
Após aumento: {'pão': 6.9, 'leite': 5.5, 'queijo': 9.0}
```